DUAL PORT RAM

DESIGN AND SYSTEMVERILOG CLASS-BASED VERIFICATION PROJECT

Complete Project Report

Project Title	Dual Port RAM Design and Verification
HDL	SystemVerilog
Simulation Platform	EDA Playground / Cadence Xcelium (as configured)
Memory Organization	16 words × 8 bits
Verification Style	Class-based, non-UVM testbench

Submitted by: _____

Register Number: _____

Department / Institution: _____

Academic Year: _____

Report Notes

This report documents a small dual-port RAM design and its class-based SystemVerilog verification environment. It is written to match the two-file EDA Playground arrangement: the synthesizable DUT is placed in **design.sv**, and the interface, verification classes and top-level testbench are placed in **testbench.sv**.

Important: The report describes the intended design and testbench behavior. A successful simulation result must only be claimed after the code has been run and the simulator log has been checked. Replace the result placeholders with your actual run details before submission.

Table of Contents

- 1. Abstract
- 2. Introduction and Definition
- 3. Problem Statement
- 4. Objectives and Scope
- 5. Dual Port RAM Fundamentals
- 6. Design Specifications
- 7. Block-Level Architecture
- 8. DUT Design Description
- 9. Verification Methodology
- 10. SystemVerilog Testbench Components
- 11. Test Scenarios and Test Plan
- 12. Simulation Procedure
- 13. Results and Discussion
- 14. Advantages, Limitations and Future Enhancements
- 15. Conclusion
- 16. References
- Appendix A. Complete RTL Code
- Appendix B. Complete Testbench Code

1. Abstract

This project presents the design and verification of a synchronous dual-port Random Access Memory (RAM) using SystemVerilog. The memory contains 16 locations, each capable of storing 8 bits, giving a total capacity of 128 bits. Two independent ports, Port A and Port B, provide address, write-enable, read-enable, input-data and output-data signals. Both ports share a common clock and reset.

A class-based, non-UVM verification environment is developed to exercise the design. The environment includes a configuration class, transaction class, driver, scoreboard, environment and test class. Randomized transactions are generated and driven into the DUT. The scoreboard maintains a reference memory model and compares expected read data with the DUT outputs. Constraints prevent ambiguous cross-port same-address conflicts when at least one port performs a write.

The project demonstrates RTL modeling, interfaces, classes, virtual interfaces, constrained randomization, concurrent testbench execution and reference-model-based checking.

2. Introduction and Definition

Random Access Memory (RAM) is a semiconductor memory in which data can be stored and retrieved using an address. In a conventional single-port RAM, only one access port is available at a time. A **dual-port RAM** provides two access ports, allowing two independent operations to be requested during the same clock cycle, subject to the memory's collision rules.

In this project, each port can independently request a read or write operation. Both ports share the same clock. The memory is modeled as an array of sixteen 8-bit words. The design uses synchronous operation: writes and read-output updates occur on the rising edge of the clock.

3. Problem Statement

The objective is to design a small memory block that supports two access ports and to verify that the memory stores and returns data correctly. The verification environment must test independent Port A and Port B operations, simultaneous activity, reset behavior and same-port read/write behavior. It must also detect mismatches between the expected data and the actual DUT output.

4. Objectives and Scope

- Design a 16 × 8-bit synchronous dual-port RAM.
- Provide independent address, control and data signals for Port A and Port B.
- Implement synchronous active-high reset.
- Model same-port simultaneous read and write as write-first behavior.
- Create a reusable class-based SystemVerilog testbench without UVM.
- Generate randomized transactions using SystemVerilog constraints.
- Compare DUT read data against a reference memory model.
- Understand the role of an interface, virtual interface, driver, scoreboard, environment and test.

Scope: The implementation is intended for functional RTL simulation and learning. It does not model physical memory timing, FPGA block-RAM inference guarantees, asynchronous clocks, byte enables, ECC or a vendor-specific dual-port RAM primitive.

5. Dual Port RAM Fundamentals

5.1 Memory organization

The declaration `logic [7:0] mem [0:15];` creates sixteen memory words. Each word is 8 bits wide. A 4-bit address selects one of sixteen locations because $2^4 = 16$.

5.2 Port signals

Signal	Width	Description
clk	1 bit	Common clock; operations are synchronized to its rising edge.
rst	1 bit	Active-high synchronous reset.
wr_en_a / wr_en_b	1 bit each	Enables a write on the corresponding port.
rd_en_a / rd_en_b	1 bit each	Enables a read on the corresponding port.
addr_a / addr_b	4 bits each	Selects one of 16 memory locations.
din_a / din_b	8 bits each	Write data supplied to each port.
dout_a / dout_b	8 bits each	Registered read data from each port.

5.3 Read and write behavior

- **Write:** When a port's write enable is high at a rising clock edge, its input data is written to the addressed memory word.
- **Read:** When a port's read enable is high, the addressed word is copied to that port's registered output.
- **Write-first on one port:** If read and write enables are both high for the same port in one cycle, the output is assigned the new input data.
- **Hold:** If a port's read enable is low, its output retains its previous value.
- **Reset:** When reset is high at a rising edge, both outputs and all memory words are cleared to zero.

The testbench constrains cross-port accesses so that two ports do not access the same address in a cycle when either access is a write. This avoids relying on unspecified or technology-dependent dual-write/read-write collision behavior.

6. Design Specifications

Parameter	Specification
Memory type	Synchronous dual-port RAM
Memory depth	16 words
Data width	8 bits
Address width	4 bits
Total capacity	128 bits (16 × 8)
Number of ports	2
Clocking	One common clock
Reset	Synchronous, active high

Parameter	Specification
Read output	Registered
Same-port read/write	Write-first
Verification style	Class-based SystemVerilog, non-UVM
Random transaction count	100 by default

7. Block-Level Architecture

The design-under-test (DUT) is connected to the testbench through a SystemVerilog interface. The test object creates a configuration object and an environment object. The environment creates a driver and a scoreboard. The transaction class defines the randomized inputs. The driver applies these values to the interface, while the scoreboard samples the interface and checks the DUT outputs against a reference model.

Component	Main responsibility
Top module	Generates clock, instantiates interface and DUT, starts the test.
Interface	Groups the signals connecting testbench and DUT.
Configuration	Stores test settings such as transaction count.
Transaction	Defines randomized control, address and data values.
Driver	Drives transaction values onto the interface.
Scoreboard	Maintains reference memory and checks read outputs.
Environment	Constructs and runs the driver and scoreboard.
Test	Creates configuration and environment, then starts execution.

Note: In this compact implementation, the scoreboard samples the interface directly. A separate monitor class and mailbox are not included. They can be added as an enhancement for a more modular verification architecture.

8. DUT Design Description

8.1 Memory declaration

The memory is declared as an unpacked array of sixteen elements, with each element eight bits wide. An integer loop variable is used to clear every location during reset.

8.2 Sequential logic

The `always_ff @(posedge clk)` block describes edge-triggered sequential behavior. Nonblocking assignments are used for the memory and output registers so that updates occur according to SystemVerilog sequential assignment semantics.

8.3 Collision policy

The DUT contains independent write statements for both ports. If both ports write the same address on the same edge, the model does not define a portable priority policy. The transaction constraint prevents this case in the supplied randomized test. The constraint also prevents one port from reading an address that the other port writes in the same cycle.

9. Verification Methodology

The testbench follows a simple class-based verification approach. It is not a UVM testbench. The transaction object represents one cycle's worth of activity on both ports. The driver applies that activity on the falling edge of the clock, giving the DUT time to sample it at the next rising edge. The scoreboard waits for the rising edge and a short simulation delay before checking registered outputs.

9.1 Why drive on the falling edge?

Driving at the falling edge separates testbench stimulus from the DUT's rising-edge sampling event. This reduces race conditions between the driver and sequential RTL in this simple testbench.

9.2 Reference model

The scoreboard contains a separate 16 × 8-bit array named `model`. It predicts read data from this array, applies write-first forwarding for same-port read/write, compares the expected value with the DUT output using case inequality (`!==`), and then updates the reference memory for writes.

9.3 Four-state comparison

The use of `!==` ensures that X or Z values are treated as mismatches rather than being silently accepted. This is useful for detecting uninitialized or unknown outputs.

10. SystemVerilog Testbench Components

Component	Explanation
Interface	Groups reset and both ports' signals. The interface is instantiated with the common clock.
Virtual interface	Allows class-based components to access the actual interface instance.
Configuration class	Stores num_transactions; default value is 100.
Transaction class	Uses rand variables for both ports and a constraint to avoid cross-port write conflicts.
Driver class	Waits for a falling edge and drives the transaction signals.
Scoreboard class	Predicts output data and reports PASS or FAIL.
Environment class	Creates driver and scoreboard and runs them concurrently using fork/join_any.
Test class	Creates configuration and environment objects and calls env.run().
Top module	Initializes signals, applies reset, creates the test object and finishes simulation.

11. Test Scenarios and Test Plan

Test ID	Scenario	Expected behavior
TC01	Reset	Both outputs and all memory locations clear to zero.
TC02	Port A write then read	Port A returns the previously written data.
TC03	Port B write then read	Port B returns the previously written data.
TC04	Both ports read different addresses	Each output returns its own addressed data.
TC05	Port A write while Port B reads a different address	Port A stores data; Port B returns its addressed old data.

Test ID	Scenario	Expected behavior
TC06	Port B write while Port A reads a different address	Port B stores data; Port A returns its addressed old data.
TC07	Same-port read and write	The corresponding output receives the new input data (write-first).
TC08	Randomized traffic	Scoreboard reports no mismatches for valid generated transactions

The supplied testbench randomizes 100 transactions by default. Randomization does not guarantee that every listed scenario occurs in a particular run. For a formal project submission, directed tests or functional coverage should be added to guarantee and measure scenario coverage.

12. Simulation Procedure

- Open EDA Playground and select SystemVerilog as the language.
- Select a SystemVerilog-capable simulator, such as the configured Cadence Xcelium simulator.
- Place the DUT module in the Design pane/file (design.sv).
- Place the interface, classes and top module in the Testbench pane/file (testbench.sv).
- Use the two-file version: do not add the old seven include statements when the complete testbench is in one file.
- Ensure the top-level module is tb_top if the platform asks for a top module.
- Save the playground and click Run.
- Inspect the compilation and simulation log. Resolve any compile errors before interpreting functional results.

Compilation note: The testbench references the DUT module by the name `dual_port_ram`. The design file must define that exact module name. The interface and classes must be compiled as part of the testbench source.

13. Results and Discussion

The testbench prints a PASS message when a sampled read output matches the scoreboard's expected value. It prints an error message when a mismatch is detected. The simulation should also print the test-started, test-completed and simulation-finished messages.

Do not report a successful result until it has been observed in your own simulator log. Complete the following table after running the project:

Item	Actual result (fill after simulation)
Simulator and version	_____
Date of simulation	_____
Number of transactions	_____
Compilation status	PASS / FAIL
Functional check status	PASS / FAIL / NOT COMPLETED
Number of scoreboard mismatches	_____
Important log messages	_____

A clean compilation confirms that the source is syntactically accepted and that the design hierarchy can be elaborated. A functional PASS requires the simulation to complete and the scoreboard to report no unexpected

mismatches. The absence of an error message alone should not be used as the only evidence; review the full log and confirm that the test reached its finish message.

14. Advantages, Limitations and Future Enhancements

14.1 Advantages

- Two ports allow independent memory access in one clocked design.
- The class-based testbench separates stimulus generation, driving and checking.
- Randomization explores a range of addresses, data values and control combinations.
- A reference model provides an independent expected-data check.
- The design is small and suitable for learning RTL and SystemVerilog verification concepts.

14.2 Limitations

- The testbench uses one common clock and does not support independent asynchronous clocks.
- A separate monitor, generator and mailbox are not implemented.
- The random test does not guarantee every test scenario in every run.
- Cross-port same-address collisions are constrained away instead of being verified.
- No functional coverage, assertions, byte enables, ECC or memory timing model is included.
- The simple RTL reset loop may not map to every FPGA or ASIC memory macro; implementation-specific memory inference should be checked.

14.3 Future enhancements

- Add a dedicated monitor class and communicate observed transactions using mailboxes.
- Add directed tests for every required read/write combination.
- Add covergroups and coverpoints for address, data, operation and port combinations.
- Add SystemVerilog assertions for protocol and control-signal checks.
- Define and test explicit same-address cross-port collision behavior.
- Parameterize data width and memory depth.
- Add support for independent clocks and configurable reset behavior.
- Convert the environment to UVM if required by the project or course.

15. Conclusion

A synchronous 16 × 8-bit dual-port RAM and a class-based SystemVerilog verification environment have been described. The design provides two independently controlled ports sharing a common clock. The verification code uses a transaction class for randomized stimulus, a driver to apply inputs, and a scoreboard reference model to check read data. Configuration, environment and test classes organize the testbench into reusable components.

The project provides practical exposure to SystemVerilog RTL, interfaces, virtual interfaces, object-oriented testbench coding, constrained randomization, concurrency and scoreboard-based functional checking. The final project result should be confirmed using a successful simulator run and the actual simulation log.

16. References

- IEEE Std 1800, SystemVerilog—Unified Hardware Design, Specification, and Verification Language.
- EDA Playground, online HDL simulator and examples: https://www.edaplayground.com/
- Cadence Xcelium Simulator documentation (for simulator-specific compile and run options).
- Course notes and laboratory material on digital design, RAM architectures and SystemVerilog verification.

Appendix A. Complete RTL Code (design.sv)

Place the following module in the Design pane/file. The top-level module name must remain dual_port_ram.

A.1 Dual Port RAM RTL

```
module dual_port_ram (
    input logic clk,
    input logic rst,

    // Port A
    input logic wr_en_a,
    input logic rd_en_a,
    input logic [3:0] addr_a,
    input logic [7:0] din_a,
    output logic [7:0] dout_a,

    // Port B
    input logic wr_en_b,
    input logic rd_en_b,
    input logic [3:0] addr_b,
    input logic [7:0] din_b,
    output logic [7:0] dout_b
);

    logic [7:0] mem [0:15];
    integer i;

    always_ff @(posedge clk) begin
        if (rst) begin
            dout_a <= 0;
            dout_b <= 0;
            for (i = 0; i < 16; i = i + 1)
                mem[i] <= 0;
        end
        else begin
            if (wr_en_a)
                mem[addr_a] <= din_a;

            if (wr_en_b)
                mem[addr_b] <= din_b;

            // Same-port read/write behavior: write-first
            if (rd_en_a) begin
                if (wr_en_a)
                    dout_a <= din_a;
                else
                    dout_a <= mem[addr_a];
            end

            if (rd_en_b) begin
                if (wr_en_b)
                    dout_b <= din_b;
                else
                    dout_b <= mem[addr_b];
            end
        end
    end
endmodule
```

Appendix B. Complete Testbench Code (testbench.sv)

Place the following code in the Testbench pane/file. This single file includes the interface, all classes and the tb_top module. Do not add separate include statements for the same classes when using this arrangement.

B.1 Complete class-based testbench

```
// ================================
// INTERFACE
// ================================
interface dual_port_ram_if(input logic clk);
    logic rst;

    logic wr_en_a, rd_en_a;
    logic [3:0] addr_a;
    logic [7:0] din_a, dout_a;

    logic wr_en_b, rd_en_b;
    logic [3:0] addr_b;
    logic [7:0] din_b, dout_b;
endinterface


// ================================
// CONFIGURATION CLASS
// ================================
class ram_config;
    int num_transactions;

    function new();
        num_transactions = 100;
    endfunction
endclass


// ================================
// TRANSACTION CLASS
// ================================
class ram_transaction;
    rand bit wr_en_a, rd_en_a;
    rand bit [3:0] addr_a;
    rand bit [7:0] din_a;

    rand bit wr_en_b, rd_en_b;
    rand bit [3:0] addr_b;
    rand bit [7:0] din_b;

    // Avoid cross-port same-address conflicts whenever
    // at least one port writes.
    constraint no_collision {
        if (wr_en_a && (wr_en_b || rd_en_b))
            addr_a != addr_b;

        if (wr_en_b && rd_en_a)
            addr_a != addr_b;
    }
endclass


// ================================
// DRIVER CLASS
// ================================
class ram_driver;
    virtual dual_port_ram_if vif;

    function new(virtual dual_port_ram_if vif);
        this.vif = vif;
    endfunction

    task drive(ram_transaction tr);
        @(negedge vif.clk);

        vif.wr_en_a = tr.wr_en_a;
        vif.rd_en_a = tr.rd_en_a;
        vif.addr_a  = tr.addr_a;
        vif.din_a   = tr.din_a;

        vif.wr_en_b = tr.wr_en_b;
        vif.rd_en_b = tr.rd_en_b;
        vif.addr_b  = tr.addr_b;
        vif.din_b   = tr.din_b;
    endtask
endclass


// ================================
// SCOREBOARD CLASS
// ================================
class ram_scoreboard;
    virtual dual_port_ram_if vif;
    bit [7:0] model [0:15];

    bit [7:0] expected_a;
```

```
    bit [7:0] expected_b;

    function new(virtual dual_port_ram_if vif);
        this.vif = vif;
    endfunction

    task run();
        integer i;

        for (i = 0; i < 16; i = i + 1)
            model[i] = 0;

        forever begin
            @(posedge vif.clk);
            #1;

            if (vif.rst) begin
                for (i = 0; i < 16; i = i + 1)
                    model[i] = 0;
            end
            else begin
                // Check outputs against the pre-write model.
                if (vif.rd_en_a) begin
                    if (vif.wr_en_a)
                        expected_a = vif.din_a;
                    else
                        expected_a = model[vif.addr_a];

                    if (vif.dout_a !== expected_a)
                        $error("PORT A FAIL: Expected=%h Actual=%h",
                               expected_a, vif.dout_a);
                    else
                        $display("PORT A PASS: DATA=%h", vif.dout_a);
                end

                if (vif.rd_en_b) begin
                    if (vif.wr_en_b)
                        expected_b = vif.din_b;
                    else
                        expected_b = model[vif.addr_b];

                    if (vif.dout_b !== expected_b)
                        $error("PORT B FAIL: Expected=%h Actual=%h",
                               expected_b, vif.dout_b);
                    else
                        $display("PORT B PASS: DATA=%h", vif.dout_b);
                end

                // Update reference model after read checks.
                if (vif.wr_en_a)
                    model[vif.addr_a] = vif.din_a;

                if (vif.wr_en_b)
                    model[vif.addr_b] = vif.din_b;
            end
        end
    endtask
endclass


// ================================
// ENVIRONMENT CLASS
// ================================
class ram_environment;
    ram_driver drv;
    ram_scoreboard sb;
    virtual dual_port_ram_if vif;
    ram_config cfg;

    function new(
        virtual dual_port_ram_if vif,
        ram_config cfg
    );
        this.vif = vif;
        this.cfg = cfg;
        drv = new(vif);
        sb  = new(vif);
    endfunction

    task run();
        ram_transaction tr;

        fork
            sb.run();

            begin
                repeat (cfg.num_transactions) begin
                    tr = new();

                    if (!tr.randomize())
                        $fatal(1, "Randomization failed");

                    drv.drive(tr);
                end

                // Allow the final transaction to be sampled.
                @(posedge vif.clk);
```

```
                #2;
            end
        join_any

        disable fork;
    endtask
endclass


// ================================
// TEST CLASS
// ================================
class ram_test;
    ram_config cfg;
    ram_environment env;
    virtual dual_port_ram_if vif;

    function new(virtual dual_port_ram_if vif);
        this.vif = vif;
        cfg = new();
        env = new(vif, cfg);
    endfunction

    task run();
        $display("========== TEST STARTED ==========");
        env.run();
        $display("========== TEST COMPLETED ==========");
    endtask
endclass


// ================================
// TOP MODULE
// ================================
module tb_top;
    logic clk;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    dual_port_ram_if intf(clk);

    dual_port_ram dut (
        .clk(clk),
        .rst(intf.rst),

        .wr_en_a(intf.wr_en_a),
        .rd_en_a(intf.rd_en_a),
        .addr_a(intf.addr_a),
        .din_a(intf.din_a),
        .dout_a(intf.dout_a),

        .wr_en_b(intf.wr_en_b),
        .rd_en_b(intf.rd_en_b),
        .addr_b(intf.addr_b),
        .din_b(intf.din_b),
        .dout_b(intf.dout_b)
    );

    ram_test test;

    initial begin
        intf.rst = 1;

        intf.wr_en_a = 0;
        intf.rd_en_a = 0;
        intf.addr_a = 0;
        intf.din_a = 0;

        intf.wr_en_b = 0;
        intf.rd_en_b = 0;
        intf.addr_b = 0;
        intf.din_b = 0;

        repeat (2) @(negedge clk);
        intf.rst = 0;

        test = new(intf);
        test.run();

        $display("========== SIMULATION FINISHED ==========");
        $finish;
    end
endmodule
```

Appendix C. Final Submission Checklist

- ■ The design.sv file contains the dual_port_ram module.
- ■ The testbench.sv file contains the interface, classes and tb_top module.
- ■ The simulator is set to SystemVerilog.
- ■ The top-level module is tb_top, where required.
- ■ The source compiles without errors.
- ■ The simulation reaches the finish message.
- ■ The scoreboard output has been reviewed.
- ■ The result table in Section 13 is filled with actual run details.
- ■ Student name, register number, institution and academic year are filled on the cover page.